

University of Toronto Scarborough
CSCC24 2024 Summer Midterm Test

Duration - 1 hour 30 minutes

Aids allowed: 1 aid sheet (letter size or A4, double-sided, don't hand in).

Solution And Marking

1:	/11
2:	/13
3:	/10
4:	/10
5:	/ 6
Total:	/50

1. Miscellaneous short questions.

- (a) [3 marks] The function `inits` generates all prefixes of a list:

```
inits :: [a] -> [[a]]
inits [] = [[]]
inits (x:xs) = [] : map (\y -> x:ys) (inits xs)
-- Example: inits [1,2,3] = [[],[1],[1,2],[1,2,3]]
```

Express `inits` in terms of `foldr` (reminder in the Appendix), i.e., define `z` and `op` in the following.

```
inits = foldr op z where
z = [[]] -- 1 mark
op x r = [] : map (x:) r -- 2 marks
```

- (b) [1 mark] Here is a function, its type, and a test case I tried:

```
foo :: (a -> b) -> [a] -> [b]
-- Test case: foo (\x -> x + 3) [10, 20, 30] = [13, 23]
```

Predict the answer of `foo (\x -> mod x 10) [4, 17, 21]`.

[4, 7]. 0.5 marks if [4, 7, 1]

- (c) [2 marks] Write a function `bar :: (a -> b) -> [a] -> [b]` that differs from both `map` and `foo` above.

Many solutions, but can only play with length, occurrences, order. Here are some incomplete examples:

```
bar f (x : _) = [f x]
bar f (x : xs) = [f x, f x] ++ bar xs
bar f (x : y : _) = [f y, f x]
```

- (d) [5 marks] Of a finite sequence $x_1, \dots, x_n$, the arithmetic mean and the geometric mean refer to the following:

$$\begin{aligned}\text{arithmetic mean} &= (x_1 + \dots + x_n)/n \\ \text{geometric mean} &= \sqrt[n]{x_1 \times \dots \times x_n}\end{aligned}$$

($\sqrt[n]{y}$ in Haskell is `y ** recip (fromIntegral n)`).

Design the following type `M`, instances, and functions (except `means`) so that `means xs` gives the tuple (arithmetic mean, geometric mean) of `xs`. There are reminders in the Appendix.

```
means :: Foldable t => t Double -> (Double, Double)
means xs = toMeans (foldMap single xs)
```

Solution:

```
data M = MkM Integer Double Double      -- 1 mark
-- Students may make up any constructor name in place of "MkM".
-- Also order of fields etc. Int instead of Integer also OK.

instance Semigroup M where
  (< >) :: M -> M -> M
  MkM n1 s1 p1 < > MkM n2 s2 p2 = MkM (n1+n2) (s1+s2) (p1*p2)      -- 1 mark

instance Monoid M where
  mempty :: M
  mempty = MkM 0 0 1          -- 1 mark

single :: Double -> M
single x = MkM 1 x x          -- 1 mark

toMeans :: M -> (Double, Double)
toMeans (MkM n s p) = (s / fromIntegral n, p ** recip (fromIntegral n))
-- 1 mark. Don't require fromIntegral, e.g., accept "s / n"
```

2. [13 marks] Symbolic Differentiation! The following type represents certain math expressions containing a variable x ; an example is also given.

```
data E = C Double          -- constant
      | X                  -- variable x
      | Add E E            -- e1 + e2
      | Mul E E            -- e1 * e2
      | Pow E Integer      -- e1 ^ k, k is an integer constant
-- Example: (4 * x + 5)^(-3) becomes Pow (Add (Mul (C 4) X) (C 5)) (-3)
```

- (a) [7 marks] Implement `diff` below to perform symbolic differentiation with respect to x . Don't worry about simplification (leave it to part (b)). The Appendix has the relevant differentiation laws.

```
diff :: E -> E
```

Solution:

```
diff (C _) = C 0          0.5 marks
diff X = C 1              0.5
diff (Add e1 e2) = Add (diff e1) (diff e2)    1
diff (Mul e1 e2) = Add (Mul e1 (diff e2))      2
                  (Mul (diff e1) e2)
diff (Pow e 0) = C 0      1
diff (Pow e k) = Mul (C (fromIntegral k)) (Mul (Pow e (k-1)) (diff e))
                  2
-- Do not require fromIntegral. C k is good enough. :)
```

- (b) [6 marks] We will implement just one example of simplification: everything of the forms $E \times 1$ and $1 \times E$ should be simplified to E . The function will be called `mid`. This cool trick may help:

```
case (foo, bar) of (pattern1, pattern2) -> ...
```

```
mid :: E -> E
```

```
-- Example: mid (Mul (C 1) (Mul X (C 1))) = X, i.e., 1*(x*1) = x
```

Solution:

<code>mid (C c) = C c</code>	0.5 marks
<code>-- Or: mid e@(C _) = e</code>	
<code>mid X = X</code>	0.5
<code>mid (Add e1 e2) = Add (mid e1) (mid e2)</code>	1
<code>mid (Pow e k) = Pow (mid e) k</code>	1
<code>mid (Mul e1 e2) = case (mid e1, mid e2) of</code>	3
<code>(C 1, e2') -> e2'</code>	
<code>(e1', C 1) -> e1'</code>	
<code>(e1', e2') -> Mul e1' e2'</code>	

Breakdown of the 3 marks for the `Mul` case:

1 for knowing to recurse into `e1`, `e2` before checking for `C 1`

2 for the rest

3. [10 marks] (Induction proof.) The following definitions are given.

```

h . g = \z -> h (g z)
map f [] = []
map f (x:xs) = f x : map f xs
sel p [] = []
sel p (y:ys) = if p y then y : sel p ys else sel p ys

```

Prove the following for all finite lists xs by structural induction:

```

sel p (map f xs) = map f (sel (p . f) xs)
Base case:                                             } 1 mark
LHS = sel p [] = []                                  }
RHS = map f [] = []                                  }

```

Induction step:

```

Let x, xt be arbitrary                                }
IH: sel p (map f xt) = map f (sel (p . f) xt)        } 1 mark
WTP: sel p (map f (x:xt)) = map f (sel (p . f) (x:xt)) }
(Even if some of the above 3 lines are omitted, can give that 1 mark
if the rest shows that clearly the student knows.)

```

Two cases: $p (f x)$ is True or False.

```

If False:                                             }
LHS = sel p (f x : map f xt)                        }
      = sel p (map f xt)                            } 4 marks
      = map f (sel (p . f) xt)                      }
RHS = map f (sel (p . f) xt)                        } by IH

```

```

If True:                                             }
LHS = sel p (f x : map f xt)                        }
      = f x : sel p (map f xt)                      } 4 marks
      = f x : map f (sel (p . f) xt)                }
RHS = map f (x : sel (p . f) xt)                    } by IH
      = f x : map f (sel (p . f) xt)                }

```

4. [10 marks] Lazy evaluation. The following definitions are given:

```
idx 0 [] = error ":"
idx 0 (x:xs) = x
idx n (_:xs) = idx (n-1) xs
inits [] = [[]]
inits (x:xs) = [] : map (x:) (inits xs)  -- (x:) means (\y -> x:ys) but shorter
map f [] = []
map f (x:xs) = f x : map f xs
g n = n : g n
```

Show the lazy evaluation steps of the following until the final answer is fully evaluated.

```
    idx 1 (inits (g 2))
->
Solution:
-> idx 1 (inits (2 : g 2))
-> idx 1 ([] : map (2:) (inits (g 2)))
-> idx (1-1) (map (2:) (inits (g 2)))
-> idx 0 (map (2:) (inits (g 2)))
-> idx 0 (map (2:) (inits (2 : g 2)))
-> idx 0 (map (2:) ([] : map (2:) (inits (g 2))))
-> idx 0 ((2:) [] : map (2:) (map (2:) (inits (g 2))))
-> (2:) []
-> 2 : []
```

(The TA may use a more realistic marking scheme than this because exceptions happen all the time. But in all cases the TA makes a fair judgment.)

```
1: inits expanded twice
1:   at the right times and no more than twice
1: g 2 expanded twice
1:   at the right times and no more than twice
1: map expanded once
1: idx 1 ([] : stuff) -> idx (1-1) (stuff)
1: (1-1) -> 0
1: idx 0 (foo : ...) -> foo
1: (2:) [] -> 2:[] but only as the last step
1: right final answer
```

5. [6 marks] Successive approximations. The following definitions are given. Calculate the approximation zs_3 .

`acc a [] = a : []`

`acc a (x:xs) = a : acc (a+x) xs`

`zs = 2 : acc 3 zs`

`zs0 = _|_`

`zs1 = 2 : acc 3 _|_`

`= 2 : _|_`

1 mark

`zs2 = 2 : acc 3 (2 : _|_)`

`= 2 : 3 : acc 5 _|_`

`= 2 : 3 : _|_`

2 marks

`zs3 = 2 : acc 3 (2 : 3 : _|_)`

`= 2 : 3 : acc 5 (3 : _|_)`

`= 2 : 3 : 5 : acc 8 _|_`

`= 2 : 3 : 5 : _|_`

3 marks

Can accept skipping some steps if you think the student knows what they are doing.

(End of questions.)

Appendix

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr op z [] = z
foldr op z (x : xt) = op x (foldr op z xt)
```

```
class Semigroup a where
  (<>) :: a -> a -> a
```

```
class Semigroup a => Monoid a where
  mempty :: a
```

```
class Foldable f where
  foldMap :: Monoid m => (a -> m) -> f a -> m
  -- and others
```

Differentiation laws:

$$\begin{aligned}\frac{d}{dx}\text{constant} &= 0 \\ \frac{d}{dx}x &= 1 \\ \frac{d}{dx}(E_1 + E_2) &= \frac{d}{dx}E_1 + \frac{d}{dx}E_2 \\ \frac{d}{dx}(E_1 \times E_2) &= E_1 \times \left(\frac{d}{dx}E_2\right) + \left(\frac{d}{dx}E_1\right) \times E_2 \\ \frac{d}{dx}E^k &= k \times E^{k-1} \times \frac{d}{dx}E \quad \text{if } k \neq 0 \\ \frac{d}{dx}E^0 &= 0\end{aligned}$$

Blank page.